



git

&



GitHub

1

## Git:

Git is a version control system.

version control system is a tool that helps to track changes in code.

### Q. Why Git?

- ① popular
- ② free & open source
- ③ fast & scalable.

### Usage:

- ① track the history
- ② collaborate

## GitHub:

website that allows developers to store and manage their code using Git.

### SETUP:

- ① `git config --global user.name "[firstname lastname]"` → Set a name
- ② `git config --global user.email "[valid-email]"` → Set an email address
- ③ `git config --global color.ui auto` → Set automatic command line coloring for Git
- ④ `git config --list` → it will display the credentials.

### SETUP & INIT:

- ① Initialize an existing directory as a Git repository  

Command: `git init`
- ② Retrieve an entire repository from a hosted location via URL  

Command: `git clone [URL]`
- ③ Show modified files in working directory, staged your next commit  

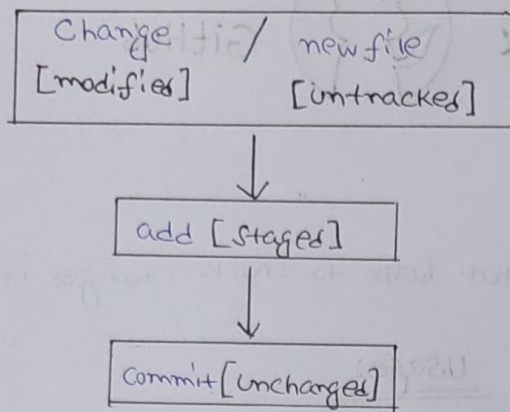
Command: `git status`

### NOTE:

Types of Status:-

- ① untracked: new file that git doesn't track yet
- ② modified: changed
- ③ Staged: file is ready to be committed
- ④ unmodified: unchanged

process:



### STAGE & SNAPSHOT:

- ① Add: adds new or changed files in your working directory to the Git staging area.

Command: `git add <file-name>`

- ② Commit: It is the record on change

Command: `git commit -m "Some message"`

- ③ Unstage a file while retaining the changes in working directory

Command: `git reset [file]`

- ④ diff of what is staged but not yet committed

Command: `git diff --staged`

- ⑤ Commit your staged content as a new commit snapshot

Command: `git commit -m "[descriptive message]"`

### BRANCH & MERGE:

- ① List your branches. a \* will appear next to the currently active branch.

Command: `git branch`

- ② Create a new branch at the current commit

Command: `git branch [branch-name]`

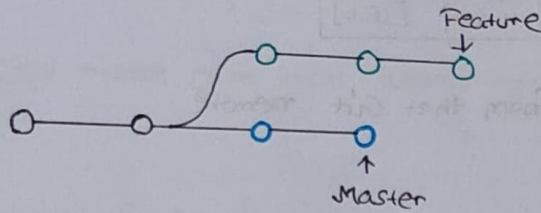
- ③ Switch to another branch and check it out into your working directory

Command: `git checkout`



## 1. What is Git Branch?

→ In Git, a branch is like a separate workspace where you can make changes and try new ideas without affecting the main project.



To rename branch

Command: `git branch -M main`

To create new branch

Command: `git checkout -b <-new branch name->`

To delete branch

Command: `git branch -d <-branch name->`

Merge the specified branch history into the current one.

Command: `git merge [branch]`

Show all commits in the current branch's history.

Command: `git log`

## INSPECT & COMPARE:

Show the commits on branch A that are not on branch B

Command: `git log branchB..branchA`

Show the commits that changed file, even across renames

Command: `git log --follow [file]`

Show the diff of what is in branch A that is not in branch B

Command: `git diff branchB..branchA`

Show any object in Git in human readable format

Command: `git show [SHA]`

## SHARE & UPDATE

4

- ① add a git URL as an alias

Command: `git remote add [alias] [URL]`

- ② fetch down all the branches from that Git remote

Command: `git fetch [alias]`

- ③ merge a remote branch into your current branch to bring it upto date

Command: `git merge [alias] / [branch]`

- ④ Transmit local branch commits to the remote repository branch

Command: `git push [alias] [branch]`

- ⑤ Fetch and merge any commits from the tracking remote branch

Command: `git pull`

## TRACKING PATH CHANGES:

- ① delete the file from project and stage the removal for commit

Command: `git rm [file]`

- ② change an existing file path and stage the move.

Command: `git mv [existing-path] [new-path]`

- ③ Show all logs with indication of any paths that moved

Command: `git log --stat -M`

## REWRITE HISTORY:

- ① Apply any commits of current branch ahead of specified one.

Command: `git rebase [branch]`

- ② Clear staging area, rewrite working tree from specified format.

Command: `git reset --hard [commit]`

## IGNORING PATTERNS:

- ① Save a file with desired patterns as gitignore with either direct string matches or wildcard globs.

Command: `logs/  
*.notes  
Pattern* /`



## Internal working of git Commands:

**NOTE:** git will not track blank folders. If you want to keep track of your folders then some files should present inside that folder.

\* Whenever you make your local repository to git repository a `.git` folder will create.

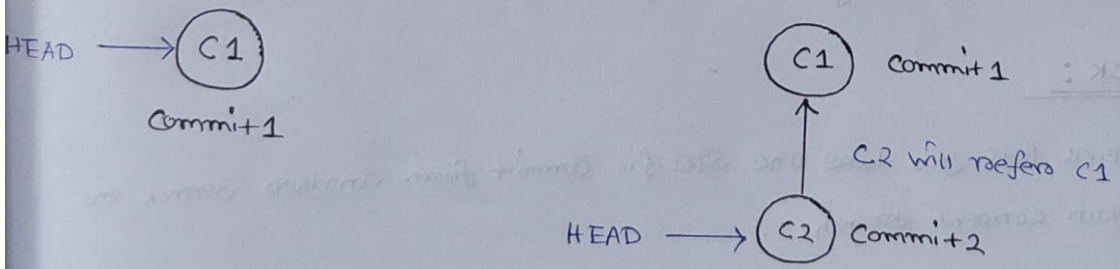
## Internal working of commit Command:

- Whenever you want to save the changes you will apply commit command.
- If you do commit, a commit object will create inside `.git` folders.
- The commit object will store 4 things -
  - ① Who has done the commit?
  - ② When commit happened?
  - ③ Commit message
  - ④ Snapshot of files in staging area → It will use an encryption algorithm called SHA1 (Secure Hashing Algorithm 1). In that the entire content of the files will converted into 40 characters alphanumeric numbers [Commit ID]

**NOTE:** branch is created when you will do commit.

When branch is created, HEAD will created. HEAD is basically pointers which points to the latest commit. We can change the position of HEAD.

Whenever you'll do second commit, it will come always from the reference of previous one, means the second commit object will not only keep it changes but also it will keep the previous commit changes. This will go on like this.



If we make changes in any tracked file, we can directly commit without separately adding them by using the command -

`git commit -a -m "Message"`

Q1. Can we do empty commit?

→ Yes we can.

`git commit --allow-empty -m "Empty commit"`

Undo the changes:

① using `git reset` command:

There are 3 attributes in `git reset` command.

① `git reset --soft commit-ID` → Changes will be back to staging area

(default)

② `git reset --mixed commit-ID` → Changes will be back to non-staging area

③ `git reset --hard commit-ID` → Changes will be deleted.

② using `git revert` command:

`git revert commit-ID`

**NOTE:** If we apply revert then new revert object will create.

`git reset` vs `git revert`:

→ If we apply `git reset --hard`, then the changes will be deleted along with all the history of that commit will also be deleted.

→ But if we apply `git revert`, then changes will be deleted but it will not remove the history.

So whenever we are working with team then `git revert` is best.



Cherry-pick:

`git cherry-pick` lets you take one specific commit from another branch and apply it to your current branch.

Example:

You are working on Branch A.

Someone else made a useful fix in Branch B (commit ID: abc123)

You don't want the whole Branch B, you just want that one fix

So we need to run —

`git cherry-pick commit-ID`



Q. Why it is called "Cherry-pick"? (7)

→ Because you're picking only the cherry (the exact commit you want), not the whole tree (full branch).

### Pull request:

A Pull request (PR) is a way to ask your team to review and approve your code changes before they are added to the main project.

### Process:

- ① You make changes in your own branch
- ② You create a Pull request.
- ③ Your team checks your code.
- ④ If everything looks good, your changes get merged into the main project.

### Forking:

Forking means making your own copy of someone else's repository on your GitHub account.

You can change your copy freely without affecting the original project.

Later if you want your changes added to the main project, you can create a pull request.